

i have one question if the text does not match exactly then what it will happend?

That's an excellent question, and it's one of the biggest challenges in NLP.

The short answer is: **a good sentiment analysis model does not need an exact text match**. It predicts sentiment based on the words and patterns it learned during training.

Let's see how this works.

## Example 1: Exact Match

Training data:

| Text              | Sentiment |
|-------------------|-----------|
| I love this phone | Positive  |

User input:

I love this phone

The model has seen this exact sentence, so it predicts **Positive** with very high confidence.

---

## Example 2: Different Sentence, Same Meaning

Training data:

I love this phone

User input:

I really love this mobile.

Although the sentence is different, it still contains important positive words:

- love
- mobile (similar to phone)

The model can still predict **Positive**.

---

### Example 3: Completely New Sentence

Training data:

```
I love this phone.
```

User input:

```
The camera quality is fantastic.
```

The model has never seen this sentence before.

However, if the vocabulary contains words like:

```
fantastic  
excellent  
great  
awesome
```

it can still predict **Positive**.

---

### Example 4: Unknown Words

Training data:

```
good  
bad  
excellent  
poor
```

User input:

```
The experience was phenomenal.
```

Suppose **phenomenal** was never in the training data.

After preprocessing:

```
experience  
phenomenal
```

If `phenomenal` is not in the vocabulary, it is ignored by a basic Bag-of-Words/TF-IDF model.

The prediction is then based on the remaining known words. If too many important words are unknown, confidence drops and the prediction may become unreliable.

---

## How Naive Bayes Thinks

Suppose the model learned these probabilities:

| Word     | Positive | Negative |
|----------|----------|----------|
| love     | 0.12     | 0.01     |
| amazing  | 0.10     | 0.01     |
| bad      | 0.01     | 0.13     |
| terrible | 0.01     | 0.15     |

User input:

I love this amazing phone

The classifier multiplies the probabilities for the known words:

```
Positive Score
=
P(Positive)
× P(love|Positive)
× P(amazing|Positive)
```

and

```
Negative Score
=
P(Negative)
× P(love|Negative)
× P(amazing|Negative)
```

The higher score wins.

Notice that the **entire sentence doesn't need to match**—only the informative words matter.

---

## When It Fails

Imagine your training data contains only:

```
good  
bad  
excellent  
terrible
```

User input:

```
Magnificent service
```

If neither `magnificent` nor `service` appears in the vocabulary, the model has almost no evidence. It may:

- produce a very low-confidence prediction,
- fall back to the class prior (the most common class), or
- make an incorrect prediction.

This is a limitation of traditional Bag-of-Words and TF-IDF approaches.